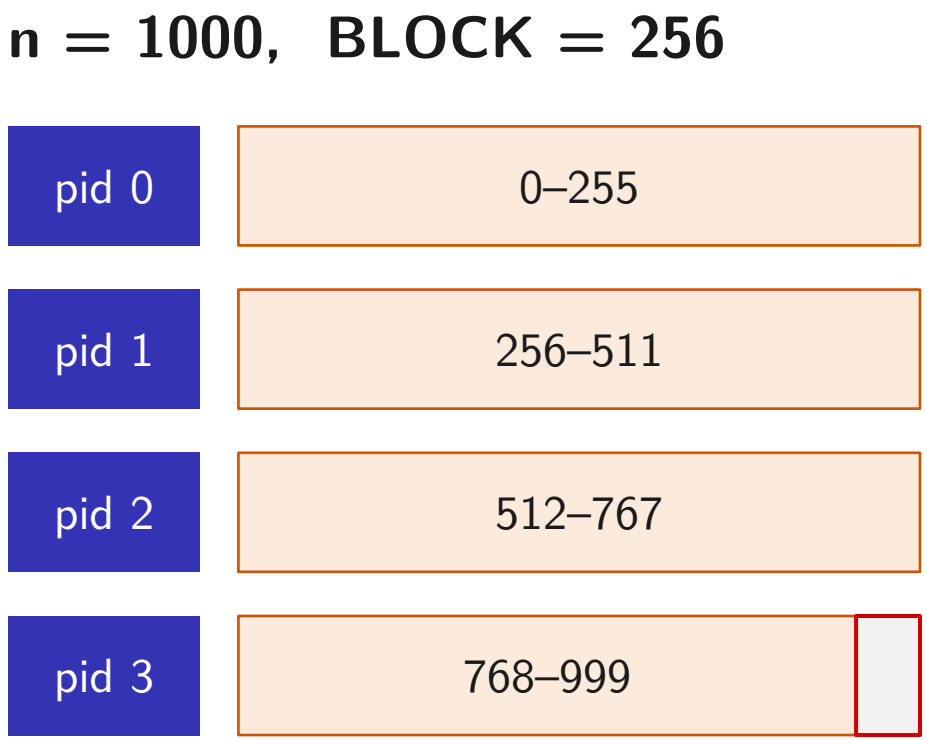
GPU Programming Model

Threads, blocks, and grids; masks and reductions in Triton; and how to test and time a kernel

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 8, Lecture 2




```
offs = pid*256 + arange(256)
mask = offs < n
```

Where we are

Last time: the hardware

SMs, warps of 32, SIMT

registers $\rightarrow$ shared $\rightarrow$ L2 $\rightarrow$ HBM

coalescing and tiling

roofline: $T \geq \max(F/\pi, Q/\beta)$



Today: the programming model

index arithmetic: who owns what

Triton blocks, masks, reductions

synchronization and streams

testing and timing a kernel

In our pipeline

T1 and Lab 3 are released after today: one Triton kernel, tested against PyTorch, timed correctly, and compared with its roofline prediction.

Learning goals for today

- Compute a thread's **global index** and size a grid with **ceiling division**.
- Write a Triton kernel with `tl.program_id`, `tl.arange`, **masks**, and block **reductions**.
- Explain when threads must **synchronize**, why blocks cannot, and what **atomics** cost.
- Describe **asynchronous** launches and **streams**; overlap copies with compute.
- **Test** a kernel against PyTorch with the right tolerances, and **time** it correctly.

A puzzle to start

```
t0 = time.perf_counter()
z = x + y                  # x, y: 10**8 floats on GPU
t1 = time.perf_counter()
print((t1 - t0) * 1e3, "ms")    # prints 0.02
```

Last lecture's roofline

$Q = 1.2$ GB, $\beta = 3.35$ TB/s

$T \geq Q / \beta \approx$ **0.36 ms**

The timer says 18× faster than the memory roof allows. What happened?

Watch out

1.2 GB in 0.02 ms would be **60 TB/s** — impossible. The launch returned **before the GPU did the work**: the timer measured the launch, not the kernel.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022).

Part 1

Threads, blocks, and grids

Which element does each thread own?

A kernel is written for one thread

```
__global__ void add(const float* x,
                    const float* y,
                    float* z, int n) {
  int i = blockIdx.x * blockDim.x
        + threadIdx.x;
  if (i < n) z[i] = x[i] + y[i];
}

int B = 256;  // threads per block
add<<<(n + B - 1) / B, B>>>(x, y, z, n);
```

- `blockIdx.x`: which block
- `blockDim.x`: threads per block
- `threadIdx.x`: which thread in the block
- `if (i < n)`: guard the tail

Key idea

Write the body for **one** element; the launch runs it for **all** of them.

Source: NVIDIA CUDA C++ Programming Guide, "Programming Model".

The global index

block 0							
0	1	2	3	4	5	6	7

threadIdx.x (inside the block)

block 1							
0	1	2	3	4	5	6	7

block 2							
0	1	2	3	4	5	6	7

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23

global index i (in the whole grid)

$$i = b \cdot B + t, \quad 0 \le t < B$$

$$B = 256, \;\; b = 2, \;\; t = 5 \quad \Rightarrow \quad i = 517$$

Key idea

Mixed radix: b = i // B and t = i mod B. Every element has **exactly one** owner.

Sizing the grid: ceiling division

$$\text{blocks} = \lceil n \;/\; B \rceil = (n + B - 1) \;//\; B$$

$$n = 1000, \quad B = 256: \quad 4 \text{ blocks} = 1024 \text{ threads}, \quad 24 \text{ idle}$$

$$n = 10^8, \quad B = 1024: \quad 97{,}657 \text{ blocks}, \quad \text{last one } 256 \text{ valid}$$

Watch out

Without the bounds check, the idle threads read and write **past the end** of the array: a crash at best, silent corruption at worst.

Choosing the block size

- A multiple of **32**: a block of 100 threads still occupies 4 warps.
- At most **1024** threads per block (CUDA limit).
- Registers and shared memory limit blocks per SM (occupancy, last lecture): start at 128–512.
- Enough blocks: $n = 10^4$, $B = 1024 \rightarrow$ only 10 blocks for 132 SMs.

Key idea

Many blocks, each a multiple of 32 threads — then **measure**; the best size depends on the kernel.

Source: NVIDIA CUDA C++ Programming Guide, technical specifications for compute capability 9.0 (H100). H100 whitepaper (132 SMs).

Grid-stride loops

```
__global__ void scale(float* x, float a,
                      int n) {
  int stride = gridDim.x * blockDim.x;
  for (int i = blockIdx.x * blockDim.x
               + threadIdx.x;
       i < n; i += stride)
    x[i] = a * x[i];
}
```

$$\text{threads} = 4 \times 132 \times 256 = 135{,}168$$

$$\text{per thread} \leq \lceil 10^8 \,/\, 135{,}168 \rceil = 740$$

- Fixed grid, **any** n.
- Each iteration: neighbours still read neighbours $\rightarrow$ **coalesced**.

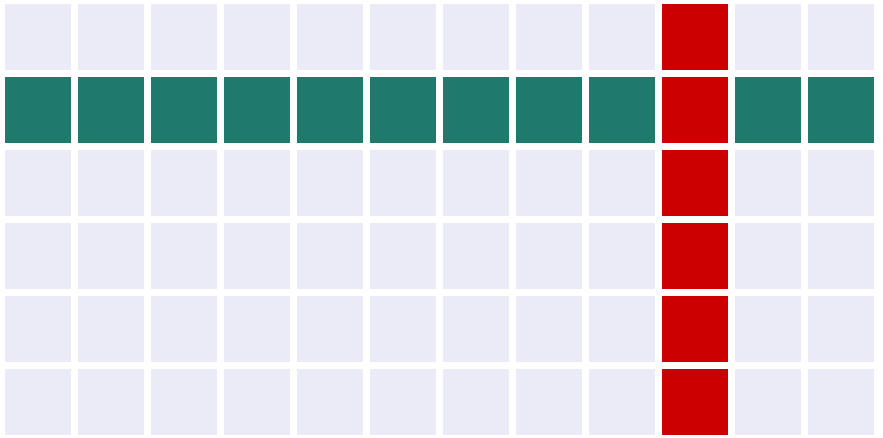
Key idea

Grid size becomes a tuning knob, not a function of n.

2-D grids: map threadIdx.x to contiguous memory

```
int col = blockIdx.x * blockDim.x
        + threadIdx.x;
int row = blockIdx.y * blockDim.y
        + threadIdx.y;
if (row < M && col < N)
  y[row * N + col] = f(x[row * N + col]);
```

row-major matrix, part of it



warp along a row: stride 1, coalesced

warp down a column: stride N, 12.5%

Key idea

threadIdx.x → the contiguous (last) dimension. Coalescing is decided by your index arithmetic.

Check yourself

Two minutes with a neighbour.

- n $= 5000$, B $= 128$: how many blocks, and how many threads are idle?
- With B $= 128$, which element does thread 37 of block 12 own?
- A kernel over a 4096×4096 row-major matrix sets `row = threadIdx.x`. Why is it slow, and what is the fix?

Part 2

Programming blocks in Triton

What if you wrote the code for a whole block at once?

CUDA's thread view, Triton's block view

	CUDA	Triton
You write code for	one **thread**	one **program** (a block)
Index	a scalar i	a vector of offsets
Shared memory, barriers	by hand	the compiler
Work per thread, vector loads	by hand	the compiler
Grid, masks, what a block does	you	**you**

Key idea

You decide **what each block computes**; Triton decides how the block's threads do it.

Source: Tillet, Kung & Cox, "Triton," MAPL 2019; Triton documentation and tutorials (triton-lang.org).

A masked vector kernel in Triton

```
import triton
import triton.language as tl

@triton.jit
def axpy_kernel(x_ptr, y_ptr, z_ptr, a, n,
                BLOCK: tl.constexpr):
    pid = tl.program_id(axis=0)
    offs = pid * BLOCK + tl.arange(0, BLOCK)
    mask = offs < n
    x = tl.load(x_ptr + offs, mask=mask)
    y = tl.load(y_ptr + offs, mask=mask)
    tl.store(z_ptr + offs, a * x + y, mask=mask)
```

- `pid` = blockIdx.x
- `offs`: the global index for all BLOCK lanes at once
- `mask`: lanes inside the array
- masked lanes are **not** read or written

Key idea

Part 1's $i = b \cdot B + t$, written for a **whole block** as a vector.

Source: Tillet, Kung & Cox, "Triton," MAPL 2019; Triton documentation and tutorials (triton-lang.org).

Launching it from PyTorch

```
def axpy(a, x, y, BLOCK=1024):
    assert x.is_cuda and x.is_contiguous()
    assert y.is_contiguous()
    assert y.shape == x.shape
    z = torch.empty_like(x)
    n = x.numel()
    grid = lambda meta: (
        triton.cdiv(n, meta["BLOCK"]),)
    axpy_kernel[grid](x, y, z, a, n,
                      BLOCK=BLOCK)
    return z
```

- grid: programs per axis
- `cdiv`: ceiling division
- tensors pass as pointers
- first call **compiles**; later calls reuse it

Watch out

The first call includes **compilation**: never time it as if it were the kernel.

Source: Tillet, Kung & Cox, "Triton," MAPL 2019; Triton documentation and tutorials (triton-lang.org).

Masks handle the tail block

program 3: offs $= 3 \cdot 256 + \text{arange}(256) = 768 \ldots 1023$ (each cell = 8 lanes)



mask true: 768 … 999 (232 lanes) **false: 24 lanes**

- Masked **load**: no memory access; the lane gets `other` (undefined if omitted).
- Masked **store**: nothing is written.

Watch out

For elementwise kernels the masked values are never stored. For **reductions** they would be summed: choose `other` with care.

BLOCK is a compile-time power of two

- `tl.constexpr`: known at compile time $\rightarrow$ each new value **compiles a new kernel**.
- `tl.arange(0, BLOCK)` needs a **power of two**: round up and mask.
- `num_warps` (default 4): BLOCK $= 1024 \rightarrow$ 8 elements per thread.

$$\text{our vocabulary: } N = 322$$
$$\text{next power of two} \geq 322: \;\; 512$$
$$512 - 322 = 190 \text{ masked lanes} \approx 37\%$$

Key idea

Power-of-two rounding wastes **less than half** the lanes, and only in the last block.

Source: Triton documentation: `tl.arange`, `triton.next_power_of_2`; course model (vocab 322).

Reductions inside a block

```
x = tl.load(p + offs, mask=mask, other=0.0)
s = tl.sum(x, axis=0)        # sum of the block

x = tl.load(p + offs, mask=mask,
            other=-float("inf"))
m = tl.max(x, axis=0)        # max of the block
```

$$\text{sum:}\quad x + 0 = x$$

$$\text{max:}\quad \max(x, -\infty) = x$$

$$\text{product:}\quad x \cdot 1 = x$$

Watch out

Masked lanes must hold the **identity** of the reduction. Forgetting `other` breaks only sizes that are **not** multiples of BLOCK.

Source: Tillet, Kung & Cox, "Triton," MAPL 2019; Triton documentation and tutorials (triton-lang.org).

Softmax overflows unless you subtract the max

$$\text{softmax}(x)[j] = \exp(x[j]) \;/\; \Sigma \exp(x[k])$$

$$= \exp(x[j] - m) \;/\; \Sigma \exp(x[k] - m), \quad m = \max x$$

$$\text{FP32 largest} \approx 3.4 \times 10^{38} \approx \exp(88.7)$$

Naive, x = [1000, 1000]
$\exp(1000) = \infty \;\rightarrow\; \infty \;/\; \infty =$ **NaN**

Stable, m = 1000
$\exp(0) \;/\; (1 + 1) =$ **0.5** each

Same value, but every exponent is ≤ 0 and the denominator is ≥ 1.

Row softmax: one program per row

```
@triton.jit
def softmax_kernel(out_ptr, in_ptr, stride, N,
                   BLOCK: tl.constexpr):
    row = tl.program_id(axis=0)
    cols = tl.arange(0, BLOCK)
    mask = cols < N
    x = tl.load(in_ptr + row * stride + cols,
                mask=mask, other=-float("inf"))
    x = x - tl.max(x, axis=0)
    num = tl.exp(x)
    y = num / tl.sum(num, axis=0)
    tl.store(out_ptr + row * stride + cols, y,
             mask=mask)
```

- grid $=$ (M,): a program per row
- BLOCK: power of two $\geq$ N
- masked lanes: $\exp(-\infty) = 0$
- the row stays **in registers**

Key idea

One read, one write per element: **fused**.

Source: Triton tutorial "Fused Softmax" (triton-lang.org).

Predicting softmax's memory traffic

FP32, M = 8192, N = 1000	Elements moved	Bytes	Q / β
Unfused: 5 PyTorch ops	8MN + 4M	262 MB	≈ 78 µs
Fused: one Triton kernel	2MN	65.5 MB	≈ 19.6 µs

$$I \approx 5\text{MN} \,/\, 8\text{MN} \approx 0.63 \text{ FLOP/byte} \ll 20 \Rightarrow \text{memory-bound}$$

$$\text{predicted speedup} \approx \text{traffic ratio} \approx 4\times$$

Prediction

Each intermediate (33 MB) fits in the 50 MB L2, so expect **less** than 4× — and explain the gap.

Source: Triton tutorial "Fused Softmax" (traffic count); H100 datasheet (SXM5), 3.35 TB/s.

Check yourself

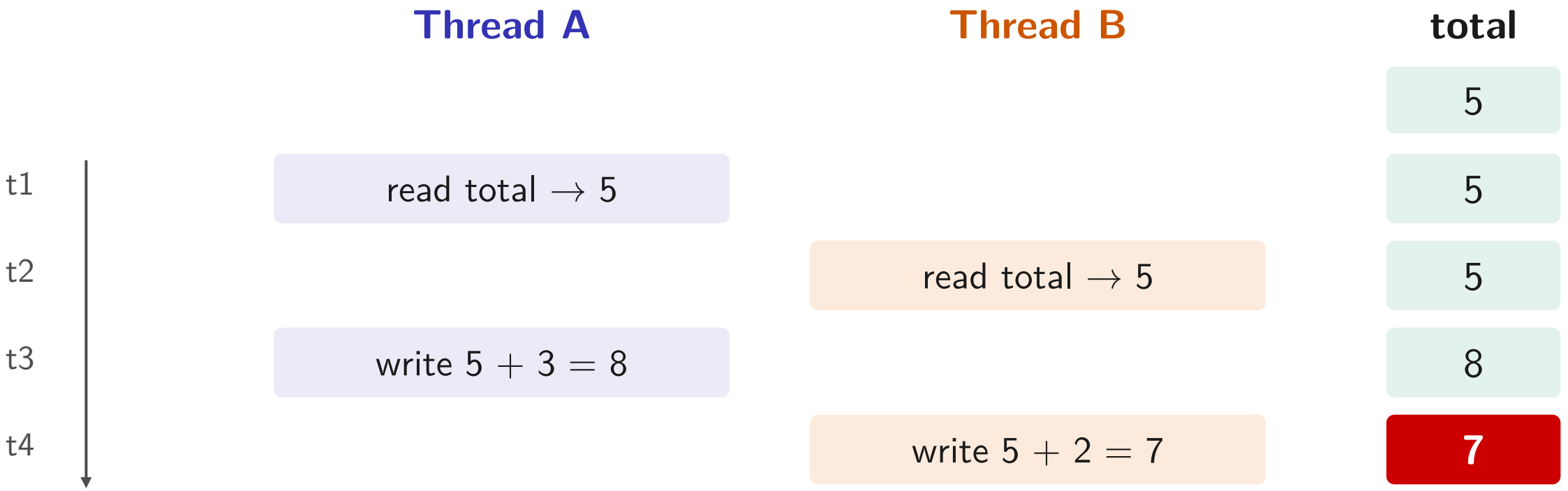
- n = 3000, BLOCK = 1024: how many programs? How many lanes are masked?
- A row-max kernel loads with `other=0.0`. For which inputs is it wrong?
- Minimum HBM traffic and time for a fused softmax over 4096 × 32,000 **BF16** logits?

Part 3

Synchronization

When must threads wait for each other — and when can they not?

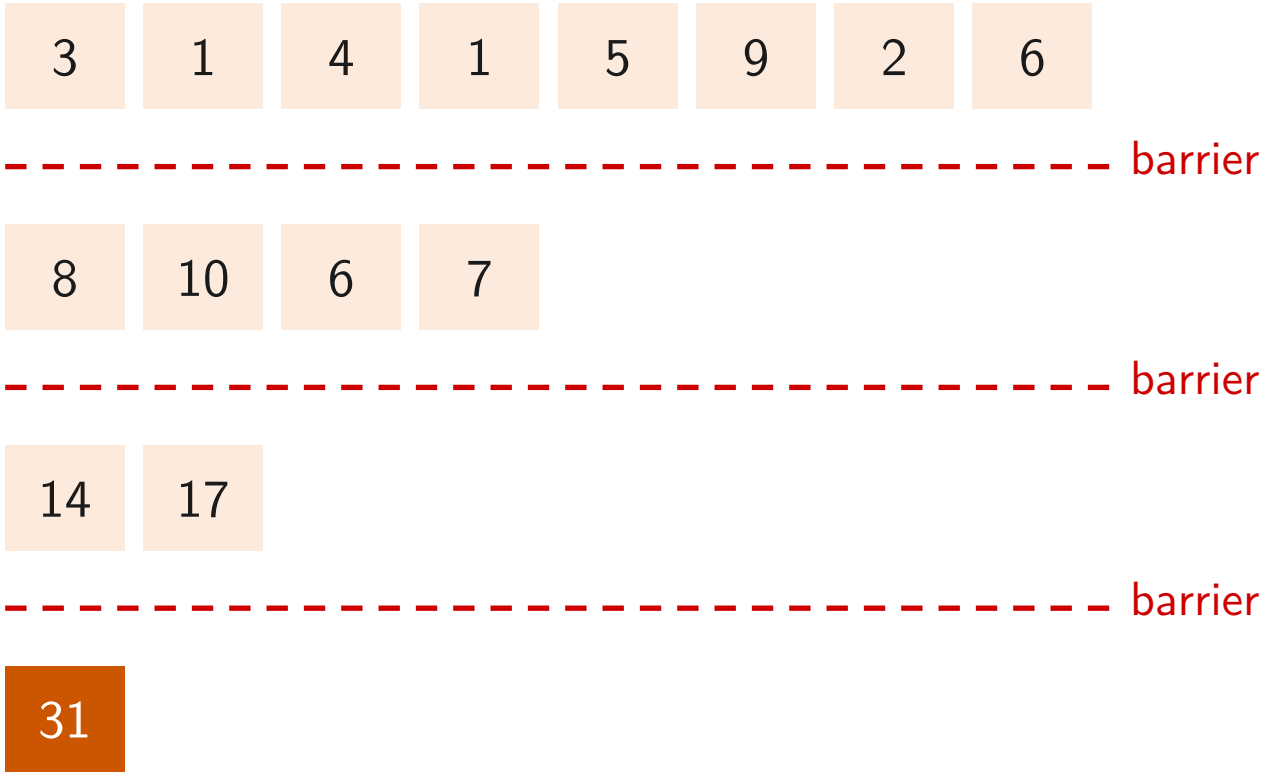
A data race: two threads, one counter



Watch out

Expected 10, got **7**: a lost update. `total += v` is read, add, write — three steps another thread can interleave.

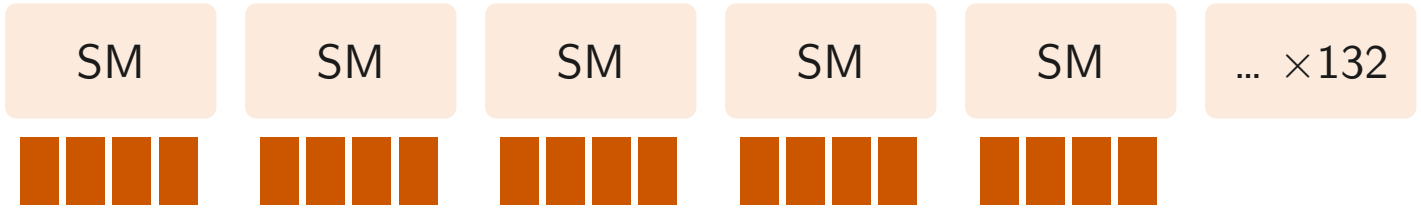
Barriers: waiting within a block



- Step k reads what **other threads** wrote in step k − 1.
- 256 values: $\log_2 256 = \mathbf{8}$ steps, 8 barriers.
- Every thread of the block must reach the barrier.
- Triton: `tl.sum` does all of this.

Source: NVIDIA CUDA C++ Programming Guide, "Synchronization Functions" (__syncthreads).

Why blocks cannot wait for each other



running: $\leq$ 8 blocks of 256 per SM $\rightarrow \leq$ 1,056 blocks



waiting: 390,625 blocks for n $= 10^8 \rightarrow \approx$ 370 waves

Grid barrier = deadlock

Running blocks would wait for blocks that cannot start until they finish.

Key idea

The only grid-wide barrier is the **end of a kernel**: "global max, then normalize" is two launches.

Source: NVIDIA CUDA C++ Programming Guide, technical specifications for compute capability 9.0 (H100).

Atomics: indivisible read–modify–write

- `tl.atomic_add(ptr, v)`: read, add, write as **one** step.
- Same-address atomics are **serialized**: one per block, not per element.
- Arrival order is **unspecified**, and float addition is not associative:

$$\text{FP32:} \quad (10^8 + 1) - 10^8 = 0, \qquad (10^8 - 10^8) + 1 = 1$$

Watch out

Atomic sums are correct but **not bitwise reproducible**. Test with tolerances; for exact repeats, sum partials in a fixed order.

Source: IEEE 754 binary32 (spacing 8 near 10^8); CUDA C++ Programming Guide, "Atomic Functions".

Summing a long vector in Triton

```
@triton.jit
def sum_kernel(x_ptr, out_ptr, n,
               BLOCK: tl.constexpr):
    pid = tl.program_id(axis=0)
    offs = pid * BLOCK + tl.arange(0, BLOCK)
    x = tl.load(x_ptr + offs, mask=offs < n,
                other=0.0)
    tl.atomic_add(out_ptr, tl.sum(x, axis=0))

out = torch.zeros(1, device="cuda")
grid = (triton.cdiv(n, 1024),)
sum_kernel[grid](x, out, n, BLOCK=1024)
```

- **In** a program: `tl.sum`
- **Across** programs: one atomic each
- 97,657 atomics for $n = 10^8$
- `out` must start at **zero**

Key idea

Deterministic: store partials, sum them in a **second launch**.

Source: Tillet, Kung & Cox, "Triton," MAPL 2019; Triton documentation and tutorials (triton-lang.org).

Check yourself

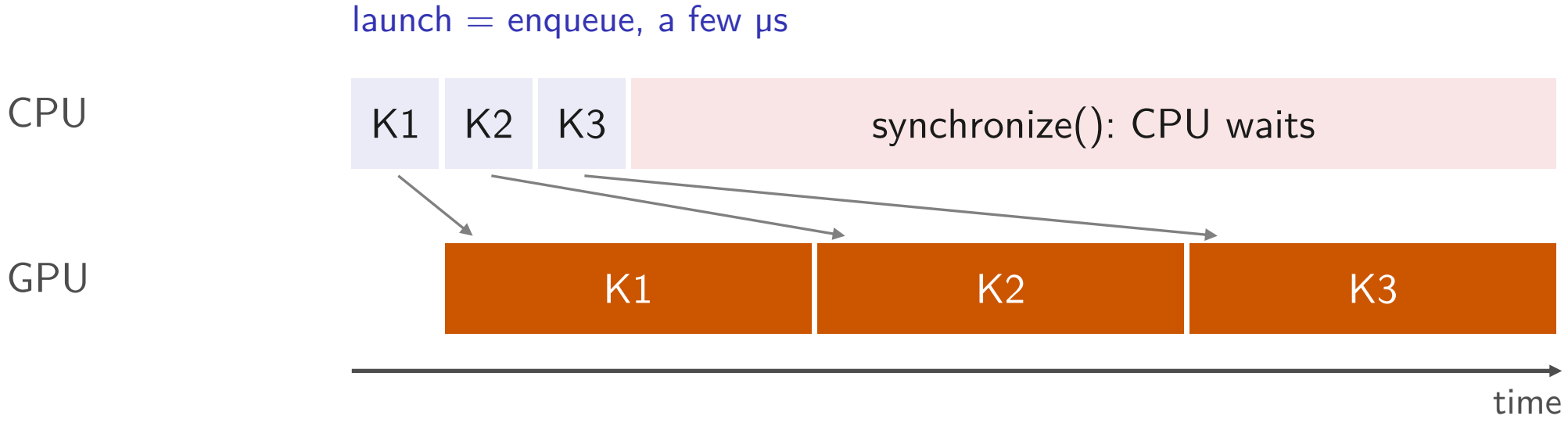
- Why does a tree reduction need a barrier between levels?
- A kernel needs the max of the whole vector before normalizing it. Why not one kernel with a grid barrier? What instead?
- Two runs of your atomic sum differ in the 7th digit. Is it a bug?

Part 4

Streams and asynchrony

When does a kernel actually run?

A launch returns immediately



Watch out

Python runs **ahead** of the GPU. A timer without synchronization measures the **launch**, not the kernel — the opening puzzle.

Source: NVIDIA CUDA C++ Programming Guide, "Asynchronous Concurrent Execution"; PyTorch "CUDA semantics".

Hidden synchronization points

Operation	Why the CPU must wait
`torch.cuda.synchronize()`	explicit: wait for all queued work
`x.item()`, `print(x)`, `x.cpu()`	needs the value on the CPU
`if loss > 0:`	Python branch on a GPU value
copy from pageable host memory	the copy must be staged first

Watch out

A stray `.item()` per step drains the queue and leaves the GPU **idle** between kernels.

Source: NVIDIA CUDA C++ Programming Guide, "Asynchronous Concurrent Execution"; PyTorch "CUDA semantics".

Streams: ordered queues of GPU work

```
s = torch.cuda.Stream()                  # 2nd queue
x_cpu = torch.randn(2**24).pin_memory()
with torch.cuda.stream(s):
    x = x_cpu.to("cuda", non_blocking=True)
y = w @ w           # default stream: overlaps
torch.cuda.current_stream().wait_stream(s)
z = 2 * x           # safe: the copy has finished
```

- **Within** a stream: in order
- **Across** streams: may overlap
- Async copy needs **pinned** memory
- `wait_stream`: order across streams

Watch out

Without `wait_stream`, the kernel could read `x` before the copy lands: a **race between streams**.

Source: NVIDIA CUDA C++ Programming Guide, "Asynchronous Concurrent Execution"; PyTorch "CUDA semantics".

Overlapping copy and compute

serial: one stream



overlapped: two streams



$$\text{serial:}\quad n\,(c+k) \;=\; 8\ \text{ms}$$

$$\text{overlapped:}\quad c + (n-1)\cdot\max(c,\,k) + k \;=\; 5\ \text{ms}$$

c: 64 MB over PCIe 5 at $\approx$ 64 GB/s $\approx$ 1 ms; k = 1 ms; n = 4 chunks

Source: PCI-SIG, PCIe 5.0 x16 $\approx$ 64 GB/s per direction; chunk sizes and kernel time are illustrative.

Check yourself

- `t0 = perf_counter(); y = f(x); t = perf_counter() - t0` — what does t measure?
- Name two operations in a training loop that silently synchronize.
- 10 chunks, copy 2 ms, compute 1 ms each: serial versus overlapped time?

IN-CLASS BUILD

Blocks, masks, and reductions

Write it, test it, time it. Work in pairs.

Exercise: a row-sum kernel

```
@triton.jit
def rowsum_kernel(x_ptr, out_ptr, N, stride,
                  BLOCK: tl.constexpr):
    row = tl.program_id(axis=0)
    cols = tl.arange(0, BLOCK)
    x = tl.load(x_ptr + row * stride + cols,
                mask=_____, other=_____)
    tl.store(out_ptr + row, _____)
```

TASKS

1. Fill the three blanks.
2. Grid and BLOCK for M $\times$ N $=$ 8192 $\times$ 1000.
3. Predict Q and the time (FP32).
4. List five test shapes.

USE

$\pi = 67$ TFLOP/s, $\beta = 3.35$ TB/s, ridge ≈ 20 FLOP/byte. Count one add per element.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022).

Exercise: solutions

```
x = tl.load(x_ptr + row * stride + cols,
            mask=cols < N, other=0.0)
tl.store(out_ptr + row, tl.sum(x, axis=0))
```

Question	Answer
Grid, BLOCK	(8192,); `next_power_of_2(1000)` $= 1024$
Q, time	$4MN + 4M \approx 32.8$ MB $\rightarrow$ ≈ 9.8 µs ($I \approx 0.25$)
Shapes	(1, 1); N $= 1023, 1024, 1025$; M $= 0$; x[:, :500]; large

Key idea

Every shape tests one line: the mask, `other`, the stride, the empty grid.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022).

What to test, and how close is close enough

dtype	rtol	atol	machine ε
float32	1.3×10^{-6}	10^{-5}	1.2×10^{-7}
bfloat16	1.6×10^{-2}	10^{-5}	7.8×10^{-3}

$$|\text{actual} - \text{expected}| \leq \text{atol} + \text{rtol} \cdot |\text{expected}|$$

- $n = 0$, $n = 1$
- $n = \text{BLOCK} - 1$, BLOCK, $\text{BLOCK} + 1$
- large, non-multiple n
- huge values (overflow)
- non-contiguous views

Key idea

Reference in **higher precision** from the same inputs; loosen a tolerance only with a stated reason.

Source: PyTorch documentation, torch.testing.assert_close (default tolerances).

A test you can run

```
SHAPES = [(1, 1), (3, 255), (3, 256),
          (3, 257), (64, 322), (0, 8)]
DTYPES = [torch.float32, torch.bfloat16]

def test_softmax():
    torch.manual_seed(0)
    for (M, N), dt in product(SHAPES, DTYPES):
        x = torch.randn(M, N, device="cuda")
        x = (100 * x).to(dt)   # overflow test
        y = softmax(x)
        ref = torch.softmax(x.float(), dim=-1)
        assert_close(y, ref.to(dt))
```

- edge shapes around BLOCK
- (64, 322): our logits
- (0, 8): empty input
- $\times$ 100: exp would overflow

Watch out

A kernel that passes only power-of-two shapes is **not** tested.

Source: PyTorch documentation, torch.testing.assert_close (default tolerances).

Timing a kernel correctly

```
def time_ms(fn, reps=50):
    for _ in range(5):
        fn()                        # warm-up, compile
    s = torch.cuda.Event(enable_timing=True)
    e = torch.cuda.Event(enable_timing=True)
    ts = []
    for _ in range(reps):
        s.record(); fn(); e.record()
        e.synchronize()             # wait for the GPU
        ts.append(s.elapsed_time(e))   # ms
    return statistics.median(ts)

ms = triton.testing.do_bench(
    fn, return_mode="median")
```

- **Warm up**: compile first
- **Events**: GPU time
- **Median** of repeats
- **Convert** to GB/s

In our pipeline

T1: a correct kernel **and** a correct harness.

Source: PyTorch torch.cuda.Event; triton.testing.do_bench (Triton source, python/triton/testing.py).

Key takeaways

- Global index $\mathbf{i = b \cdot B + t}$; grid $= \lceil \mathbf{n} \ / \ \mathbf{B} \rceil$ blocks; guard the tail with $\mathbf{i < n}$.
- Map **consecutive threads to contiguous memory**: coalescing is your index choice.
- Triton programs a **block**: `program_id`, `arange`, **masks**, `tl.sum` / `tl.max`.
- Masked lanes hold the reduction's **identity**; softmax **subtracts the max**.
- Barriers work **within a block**; across blocks, use **atomics** or a **second kernel**.
- Launches are **asynchronous**: warm up, use **events**, report the **median**.

Next time, and readings

When	What
After this lecture	T1 and Lab 3 released: a Triton kernel, tested and timed
Week 9, Lecture 1	Framework fundamentals: tensors, storage and layout, autograd, dispatch
Lab 3 (Weeks 8–10)	Triton kernel, eager vs compiled trace, JAX equivalent

READINGS

CUDA C++ Programming Guide: execution hierarchy, memory, synchronization, streams.

Tillet, Kung & Cox, "Triton: an intermediate language and compiler for tiled neural network computations," MAPL 2019.